

At least two minutes each.

Five complete speeches aligned to the same 10 slides.

Each speaker presents two slides, with a written transition and handoff.

Total speaking time: 12:35-15:29 minutes, plus questions.

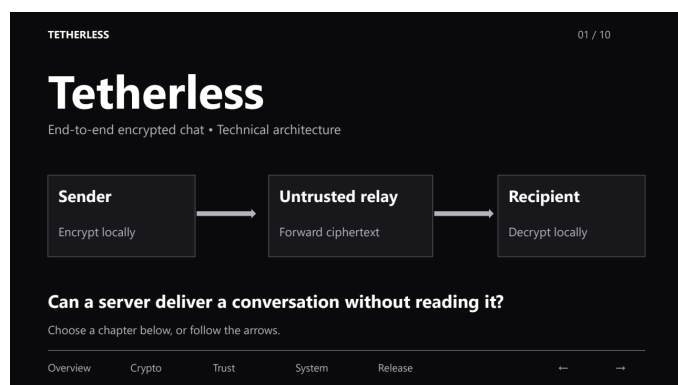
Estimates use 130-160 words per minute. Pauses and cues are extra.

Speaker / focus	Slides	Words	Time
Speaker 1 Architecture	01-02	388	2:26-3:00
Speaker 2 Cryptography	03-04	404	2:32-3:07
Speaker 3 Identity & security	05-06	401	2:31-3:06
Speaker 4 Relay & clients	07-08	411	2:35-3:10
Speaker 5 Verification & release	09-10	408	2:33-3:09

Rehearsal instructions

Read the paragraphs marked "Spoken script" aloud. The visual cues are reminders, not part of the speech. Change slides at the page break. Each speaker has more than two minutes of script even at 160 words per minute; time a rehearsal to account for your own pace.

01 Tetherless



THIS SLIDE

192 spoken words

1:12-1:29 minutes

Visual cue

Trace sender, relay, and recipient.

SPOKEN SCRIPT

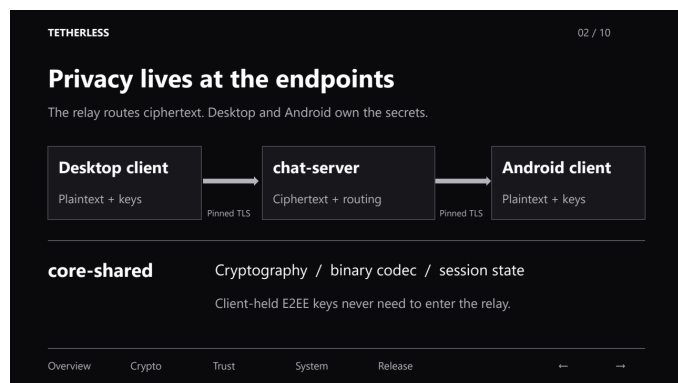
Good morning, everyone. Our project is Tetherless, an end-to-end encrypted chat system. The question behind it is simple: can a server deliver a conversation without being able to read what people are saying? That question shapes the architecture you see on this slide.

Follow the message from left to right. The sender encrypts it on their own device. The relay receives ciphertext and forwards it to the recipient. The recipient then decrypts it locally. The relay is needed for delivery, but it does not need the clients' message encryption keys. We design around the possibility that the relay could be compromised, rather than making its honesty a requirement for protecting message content.

For example, imagine sending a meeting location. The server needs enough routing information to reach the other person, but it does not need to know the location itself. That separation is the main idea of the project. Our current implementation still uses a central relay, so we describe it as relay-based encrypted messaging. As we continue, we will explain the key exchange, identity checks, delivery system, and evidence needed before release. First, let us look at where those responsibilities live.

Change to slide 2. Continue with the next page.

02 Privacy lives at the endpoints



THIS SLIDE

196 spoken words

1:14-1:31 minutes

Visual cue

Point to the client keys and the shared layer.

SPOKEN SCRIPT

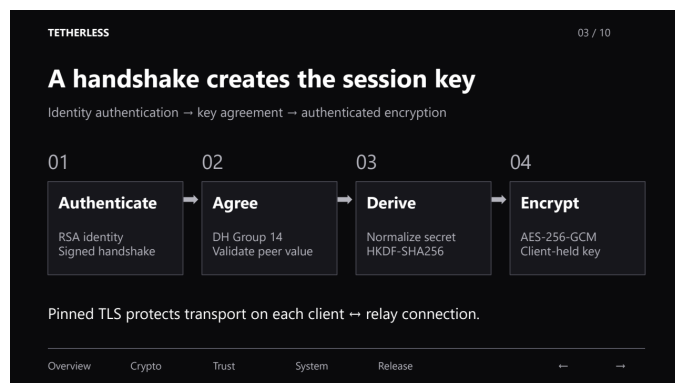
This diagram shows the same conversation through the software modules. On the left is the desktop client, in the middle is chat-server, and on the right is the Android client. Plaintext and the keys used for end-to-end encryption belong at the clients. The server handles ciphertext and the routing information required for delivery.

Both clients use core-shared for cryptography, the binary message codec, and session state. Sharing that logic reduces the risk of the desktop and Android implementations interpreting the protocol differently. It also gives us a common place to test those important behaviors. The interfaces and local storage can differ, while the conversation follows the same rules.

There is also pinned TLS on each connection between a client and the relay. Think of this as an additional transport layer. End-to-end encryption protects the message contents across the whole route; TLS protects the individual network connections. The distinction matters because the relay still needs to see routing information. We are separating responsibilities, not claiming that the server sees nothing at all. That establishes the architecture. I will now hand over to Speaker Two, who will explain how the clients establish and use a shared session key.

Handoff complete. Speaker 2 begins on slide 3.

03 A handshake creates the session key



THIS SLIDE

196 spoken words

1:14-1:31 minutes

Visual cue

Follow authenticate, agree, derive, and encrypt.

SPOKEN SCRIPT

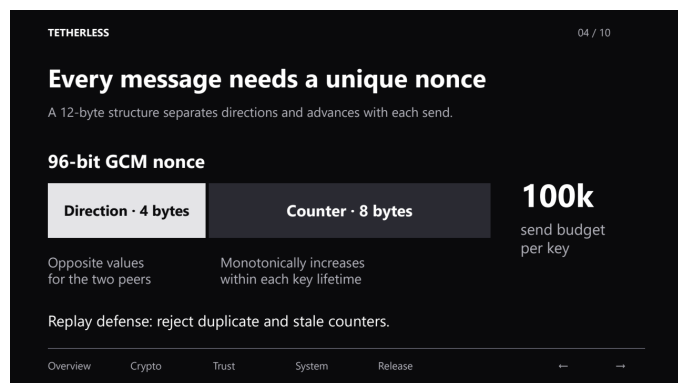
Thank you. The previous slides showed where encryption happens. This slide explains how the clients obtain the key they need. There are four stages: authenticate, agree, derive, and encrypt. Keeping these stages separate helps us understand what each one contributes to the conversation.

First, identity keys and signed handshake messages let a client check which key is participating. Next, the clients exchange and validate Diffie-Hellman public values using Group Fourteen. This gives them the material needed to establish a shared secret without sending that secret as an ordinary message through the relay. Validating the received value is part of accepting the exchange safely.

The shared secret is then normalized to a consistent byte length and passed through HKDF with SHA-256. Normalization matters because different platforms must supply the same bytes to key derivation. The resulting key is used with AES-256-GCM for authenticated encryption of message contents. The relay forwards the exchange, but it does not receive the resulting end-to-end key. This diagram is the conceptual flow, rather than every protocol message. Once that key exists, another detail becomes critical: how we give each encrypted message a distinct nonce. That is the focus of the next slide.

Change to slide 4. Continue with the next page.

04 Every message needs a unique nonce



THIS SLIDE

208 spoken words

1:18-1:36 minutes

Visual cue

Point to direction, counter, and the send budget.

SPOKEN SCRIPT

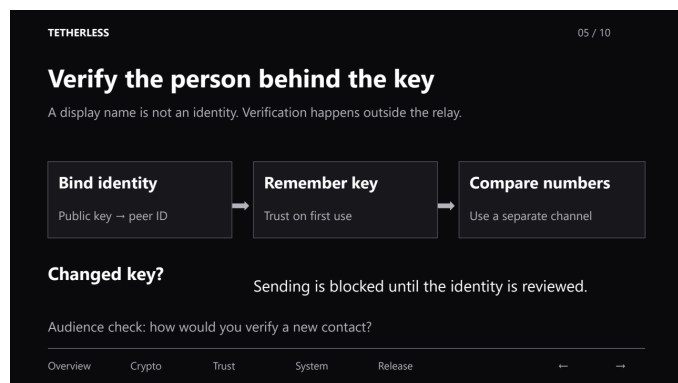
A session key is reused for multiple messages, so encryption also needs a nonce that does not repeat under that key. Our nonce is twelve bytes long. As the diagram shows, four bytes identify the direction and eight bytes hold a counter. These fields solve two different parts of the uniqueness problem.

The counter advances as one side sends messages. However, a counter alone would not separate the two senders: both peers could begin with the same counter value. The direction field prevents that collision. Its value follows the ordering of the peer identifiers, so the two sides choose opposite directions. It is not assigned according to who happened to begin the handshake.

The implementation also limits each key to a send budget of one hundred thousand messages and starts renewal when that budget is exhausted. That number is a key lifetime policy, not a claim that the current IV test sends one hundred thousand messages. Separately, replay checks reject duplicate and stale counters, so a previously captured message cannot simply be accepted again as new. The important idea is that key agreement, nonce uniqueness, and replay handling work together. I will now hand over to Speaker Three to explain how users establish trust in a contact's identity.

Handoff complete. Speaker 3 begins on slide 5.

05 Verify the person behind the key



THIS SLIDE

200 spoken words

1:15-1:33 minutes

Visual cue

Follow the three trust steps; look up for the audience question.

SPOKEN SCRIPT

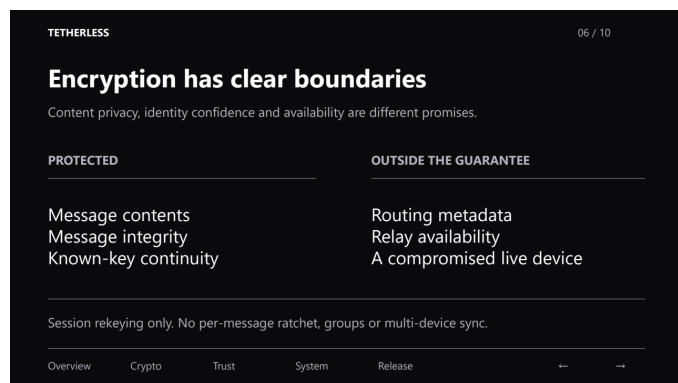
Thank you. Encrypting a message is useful only if we have confidence about the person receiving it. A display name is not enough for that purpose. Someone can choose a familiar name, so Tetherless connects a peer identifier to an identity public key instead of treating a name as proof.

The three stages on this slide describe the trust process. First, bind the identity to the key. Second, remember the key through trust on first use. Third, compare safety numbers through a trusted separate channel. Remembering the first key helps detect a later change, but it does not, by itself, prove who was behind that first contact.

For example, if I add a teammate, I can compare the safety number with them in person or through another channel that I already trust. The purpose is to connect the cryptographic identity to the human I intend to contact. If the key later changes, sending is blocked until the identity is reviewed. A change is a reason to investigate, not something to silently accept. A useful question for this audience is: how would you verify a new contact? Keep that question in mind as we look at the limits of the protection.

Change to slide 6. Continue with the next page.

06 Encryption has clear boundaries



THIS SLIDE

201 spoken words

1:16-1:33 minutes

Visual cue

Compare the protected and exposed columns.

SPOKEN SCRIPT

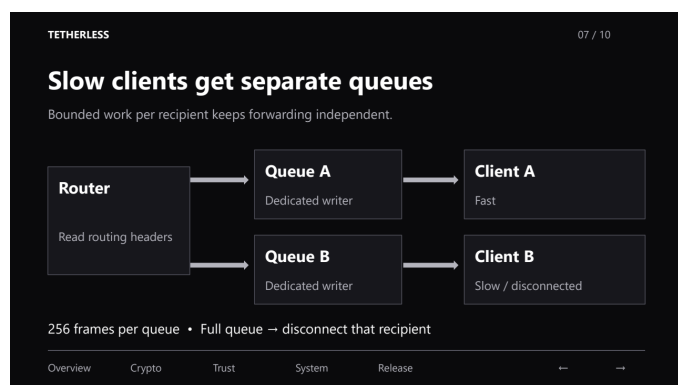
This slide separates the protections we aim to provide from the risks that remain. On the left are message contents, message integrity, and continuity of a known key. These depend on correct implementation and on the endpoints remaining trustworthy. On the right are things that end-to-end encryption does not automatically solve.

The relay still sees routing information, including which parties communicate, the timing of traffic, and frame lengths. It can also refuse to deliver messages. For example, hiding the meeting location inside an encrypted message does not hide the fact that two people are communicating. Content privacy and service availability are different properties, and both need to be discussed honestly.

There is also the endpoint boundary. If an attacker controls a live device while messages are being read, protecting network traffic cannot make that device trustworthy again. Our current design renews session keys, but it does not include a per-message Double Ratchet. Group messaging and multi-device synchronization are also outside the current scope. These limits help us choose what to test and explain what users can reasonably expect. With the security boundaries established, I will hand over to Speaker Four to explain how the relay and clients handle delivery in practice.

Handoff complete. Speaker 4 begins on slide 7.

07 Slow clients get separate queues



THIS SLIDE

207 spoken words

1:18-1:36 minutes

Visual cue

Trace the fast and slow recipient paths.

SPOKEN SCRIPT

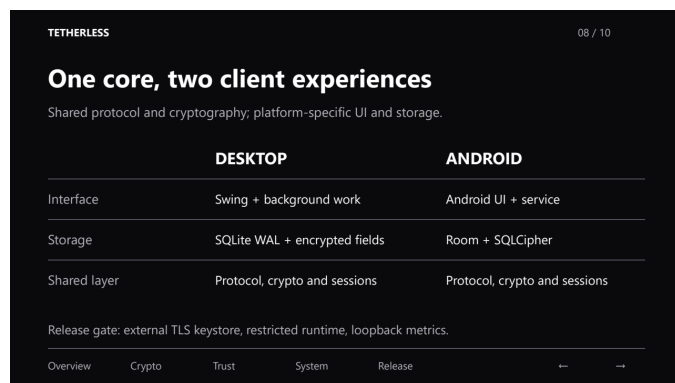
Thank you. Security is only one part of a usable chat system. Delivery must also behave sensibly when clients have different network conditions. Imagine that Client A has a good connection while Client B is slow or temporarily disconnected. We do not want the slow connection to hold up unrelated recipients.

The diagram shows how the relay separates that work. The router reads the routing headers and places frames into the appropriate recipient queue. Each recipient has a dedicated writer, so one writer waiting on its client does not stop another writer from progressing. The queues are bounded at two hundred and fifty-six frames. If a queue fills, the affected connection is closed rather than allowing work to accumulate without a limit.

Connection handling includes other safeguards as well. Conditional registry removal prevents cleanup from an old connection from removing a newer connection for the same client. Heartbeats help identify dead peers. At the input boundary, binary framing checks lengths before allocation instead of accepting unrestricted data. Together, these choices keep routing work controlled and failures more localized. The relay still routes ciphertext; it does not need to decrypt the message to perform these tasks. Next, we can compare how the two clients use that shared system.

Change to slide 8. Continue with the next page.

08 One core, two client experiences



The screenshot shows a presentation slide with a dark background. At the top left is the 'TETHERLESS' logo and at the top right is '08 / 10'. The title 'One core, two client experiences' is centered, followed by the subtitle 'Shared protocol and cryptography; platform-specific UI and storage.' Below this is a table with two columns: 'DESKTOP' and 'ANDROID'. The table has three rows: 'Interface', 'Storage', and 'Shared layer'. The 'Shared layer' row is highlighted. Below the table is a note about the 'Release gate' and a navigation bar at the bottom with links for 'Overview', 'Crypto', 'Trust', 'System', and 'Release'.

	DESKTOP	ANDROID
Interface	Swing + background work	Android UI + service
Storage	SQLite WAL + encrypted fields	Room + SQLCipher
Shared layer	Protocol, crypto and sessions	Protocol, crypto and sessions

Release gate: external TLS keystore, restricted runtime, loopback metrics.

Overview Crypto Trust System Release

THIS SLIDE

204 spoken words

1:17-1:35 minutes

Visual cue

Move across the desktop and Android columns.

SPOKEN SCRIPT

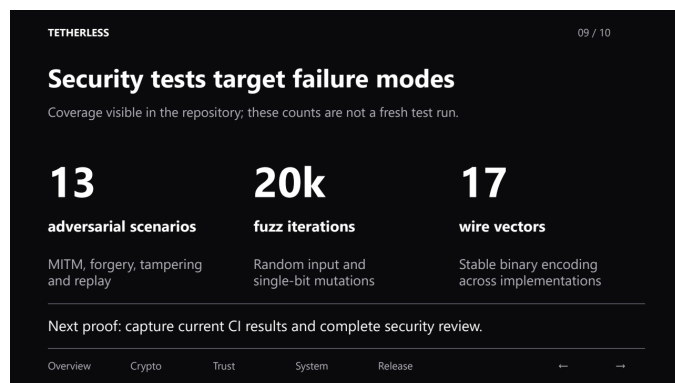
The desktop and Android applications share the protocol and cryptographic logic, but their user interfaces and storage need to fit their platforms. This table highlights those differences without changing the rules of the encrypted conversation. The shared layer is what keeps the two clients speaking the same protocol.

On desktop, the interface uses Swing. Network and cryptographic work run away from the event dispatch thread so that these operations do not occupy the thread responsible for updating the interface. Local storage uses SQLite in WAL mode, with encrypted message fields. On Android, Room provides the database access layer, SQLCipher protects the database, and AndroidKeyStore participates in protecting key material. A service supports the connection lifecycle on the mobile side.

Deployment configuration matters too. Release packages require an external TLS keystore, rather than quietly relying on a development certificate. Restricted runtime privileges and loopback-only metrics reduce unnecessary exposure around the relay. These measures complement the messaging protocol: a sound design still needs careful configuration when packaged and deployed. We have now covered the path from the protocol to the running applications. I will hand over to Speaker Five, who will explain what the repository tests cover and how we would move toward an MVP release.

Handoff complete. Speaker 5 begins on slide 9.

09 Security tests target failure modes



THIS SLIDE

201 spoken words

1:16-1:33 minutes

Visual cue

Explain the three test categories from left to right.

SPOKEN SCRIPT

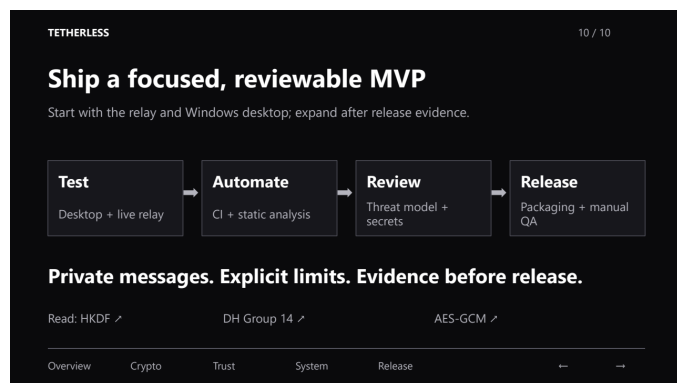
Thank you. We should evaluate a security design by the failures it handles, as well as by successful message delivery. The repository includes thirteen adversarial relay test methods, twenty thousand codec fuzz iterations, and seventeen wire vectors. The three figures on this slide describe different kinds of coverage.

Adversarial tests examine behavior such as key substitution, forged messages, ciphertext tampering, and replay. For example, changing an encrypted message should result in rejection rather than silently displaying altered content. Codec fuzzing challenges the parser with unexpected inputs. The twenty thousand iterations comprise two loops of ten thousand: one for random input and one for single-bit mutations. The wire vectors provide fixed examples of binary encoding so implementations can be checked against the same expected representation.

These categories help us ask specific questions: does the client reject an attack, does the parser handle malformed data predictably, and do both platforms interpret messages consistently? The numbers describe tests defined in the repository. They are not a report of a fresh passing run, and they do not substitute for independent security review. Before release, we need current results from the intended build and environment. That leads directly to the delivery sequence on our final slide.

Change to slide 10. Continue with the next page.

10 Ship a focused, reviewable MVP



THIS SLIDE

207 spoken words

1:18-1:36 minutes

Visual cue

Follow the release stages, then look up for the closing invitation.

SPOKEN SCRIPT

Our immediate release target is a focused MVP containing the relay and Windows desktop client. The sequence on this slide is test, automate, review, and release. Each stage should produce evidence that makes the next decision easier, rather than treating packaging as proof that the system is ready.

First, we need desktop and live relay integration checks that exercise a complete conversation. Next, CI and static analysis should make those checks repeatable for the build we plan to distribute. Security review should revisit the threat model, identity behavior, key handling, and release configuration. Packaging and manual QA then verify the experience that a user will actually install and run. For example, we should check reconnect behavior, clear failure messages, and the requirement for an external TLS keystore.

Later capabilities such as federation, group messaging, and per-message ratcheting require their own protocol work and verification. Keeping the initial scope focused gives us a clearer set of claims to support. The main takeaway is that Tetherless puts message privacy at the endpoints, states its limitations, and uses evidence to guide release decisions. Thank you for listening. We welcome questions, especially about which risk you would validate first. We can use the chapter links to return to the relevant diagram.

Open Q&A.; Use the presentation chapter links to revisit a diagram.